Drishti — Concepts, Learning Guide & Interview Prep

Every concept used in the project, explained from the ground up • what to learn • the key code, annotated. Compiled 19 July 2026.

1. The big picture — what you built

Drishti is a **full-stack 3D web application** with two halves that talk over HTTP:

• **Backend (Python / FastAPI):** receives a CAD floor-plan PDF, reads its vector drawing commands, figures out the real-world scale, finds walls / doors / windows / columns / stairs / rooms, and builds a 3D model file (GLB). Think of it as a *machine that reads architect drawings the way a junior engineer would* — except with geometry rules instead of eyes.

• **Frontend (React + three.js):** a browser app that uploads the PDF, receives the GLB, and renders it like a small video game — materials, sunlight, shadows, sky, and a camera you can fly around or walk inside with.

The data flows in one line: **PDF** → **(parse) scene.json** → **(build) scene.glb** → **(render) 3D on screen**. Everything in this report hangs off that pipeline.

2. Backend concepts (Python)

2.1 Vector PDFs vs raster images — the foundation of the whole project

A PDF exported from AutoCAD is not a picture: it is a **list of drawing instructions** — "line from point A to B, on layer WALLS, 0.5pt thick". A scanned/photographed plan is just pixels. Drishti's core bet: read the instructions directly (exact coordinates, layer names, live text) instead of guessing from pixels. The library used is **PyMuPDF (fitz)**: `page.get_drawings()` returns every line/rect with its layer, `page.get_text("words")` returns every text string with its position. **Learn:** what vector graphics are, PDF content streams (conceptually), PyMuPDF basics.

2.2 Scale detection — computational geometry + voting

The PDF is in "points" (1/72 inch on paper); the building is in feet. The bridge is the **dimension text** architects write on plans (like 12'-6"). The algorithm: find every ft-in text token with a regex, find every line it sits on top of, and compute *line length ÷ stated value = points-per-foot*. Each pairing is one **vote**; the most common vote wins, and the median of agreeing votes becomes the scale. Voting makes it robust: a few wrong pairings can't beat dozens of correct ones. This is the same idea as RANSAC / Hough voting in computer vision. **Learn:** regex, dot product for projecting a point onto a line, median vs mean, why voting beats trusting one measurement.

2.3 Rasterization + morphology — turning lines into solid walls

Wall layers give *centrelines*, but a building needs solid wall *bands* with thickness. The trick: draw the lines onto an image grid at a fixed pixels-per-foot, then use **OpenCV morphology** — **dilate** (fatten every line) then **erode** (shrink back), which merges the two faces of a wall into one solid band and closes small gaps. A key project lesson: TWO masks are kept — a deliberately fat one for door-gap detection (which is sensitive to thickness) and a clean square-cornered one for the final output. **Learn:** binary images, dilation/erosion/closing, structuring elements (RECT vs ELLIPSE), connected components.

2.4 Rooms — connected components + distance transform

A room = an enclosed pocket of free space. Algorithm: take the final wall mask, flood the *empty* space, and find **connected components** (islands of connected pixels). Any island touching the image border is "outside"; the rest are rooms (if ≥ 28 sqft). The walk-inside beacon is placed at the room's **distance-transform maximum** — the pixel farthest from every wall — not the centroid, because an L-shaped room's centroid can sit inside a wall. **Learn:** flood fill, connectedComponents, cv2.distanceTransform, centroid vs pole-of-inaccessibility.

2.5 3D construction — extrusion and the GLB format

Walls are 2D polygons; the 3D step is **extrusion**: push each polygon straight up by the wall height (~3 m). Door/window openings are subtracted as boxes before extrusion. The result is packed with **trimesh** into **GLB** — the

binary form of glTF, the "JPEG of 3D": one file holding meshes, colours and names, readable by three.js, Blender, Windows 3D Viewer. Two hard-won rules: every mesh gets a **name** (floor, wall_3, glass_0) so the viewer can assign materials by name; and **normals** (per-vertex "which way does this face point" vectors) must be exported — without them lighting math divides by zero and meshes render solid black. **Learn:** meshes = vertices + triangles, normals, the glTF/GLB format, trimesh basics.

2.6 The API layer — FastAPI + defensive engineering

FastAPI exposes the engine over HTTP: /health (is it up?), /scene (parse → JSON), /scene.glb (parse → 3D file). The production-hardening concepts — the part interviewers care about most:

• **async + threadpool:** the web server is async (one process juggles many requests), but parsing is heavy CPU work that would freeze it — so heavy calls run in a thread pool via `run_in_threadpool`.
• **Semaphore:** at most N parses at once; extra requests queue instead of racing into out-of-memory.
• **Timeout:** a wall-clock limit (120 s) converts a hung parse into a clean 504 instead of an eternal spinner.
• **Correct HTTP codes:** 413 file too large, 415 wrong type, 422 unreadable, 503 feature off (photo beta), 504 timeout.
• **CORS:** browsers block a site on domain A from calling an API on domain B unless the API sends "Access-Control-Allow-Origin". That is exactly what ALLOWED_ORIGINS configures on Render.
• **Graceful degradation:** the AI photo path (torch) is imported lazily in a try/except — the server boots and serves CAD PDFs even when AI libraries are absent, answering photos with a friendly beta message.
• **Temp-file race fix:** each /scene.glb request writes a unique temp file (mkstemp) deleted after sending — a fixed filename let two simultaneous users download each other's building.
Learn: async/await vs threads, semaphores, HTTP status codes, CORS, race conditions.

3. Frontend concepts (React + three.js)

3.1 React + TypeScript + Vite

The UI is **React** (components + state: "when data changes, re-render what depends on it"), typed with **TypeScript** (catches wrong shapes at compile time — e.g. a Room always has x, y), bundled by **Vite**. One subtle Vite concept you used: `import.meta.env.VITE_API_BASE` is a **build-time** variable — it is baked into the JavaScript at build, which is why changing it on Vercel requires a redeploy. **Learn:** components, props/state, useEffect/useRef, build-time vs runtime configuration.

3.2 three.js and react-three-fiber — the 3D scene

three.js is the browser 3D engine: a **scene graph** (tree of objects), a **camera**, and a WebGL renderer drawing 60 frames/second on the GPU. **react-three-fiber** lets you write that scene as React components (<Model />, <PlanLights />), and **drei** adds ready-mades (Sky, OrbitControls, useGLTF). **Learn:** scene graph, mesh = geometry + material, perspective camera, the render loop.

3.3 PBR materials + the boxUV trick — the most original frontend code

Realistic surfaces need **UV coordinates** (a map from 3D surface to 2D texture). Our GLB prisms have none — textures would smear. Fix: **box projection** — for each triangle, look at its normal's dominant axis (is this face pointing up, along X, or along Z?) and use the other two coordinates as UVs. It is a CPU-side "triplanar mapping", exact for axis-aligned masonry, no custom shaders. Textures themselves are **drawn procedurally on canvases** (plaster blotches, 600 mm tiles with grout lines, concrete speckle) — zero downloads, works offline. **Learn:** UV mapping, PBR (roughness/metalness), triplanar mapping, canvas 2D API.

3.4 Lighting and shadows

One directional "sun" + hemisphere sky light + tiny ambient. Two real-world lessons baked into the code: the shadow camera **frustum** must cover the whole building (the default ±3.5 m box silently clipped shadows on a 14 m plan), and shadow **bias/normalBias** must be tuned on a real GPU — the cloud's software renderer hid banding that the user's GPU showed. Interiors get a **point light riding the camera** (a "lantern") because rooms are in sun-shadow. **Learn:** light types, shadow mapping, bias artifacts (acne/peter-panning), frustum.

3.5 Camera animation — GSAP tweening

Walk-inside glides tween **camera position and orbit target together** over 0.8 s with an ease-out curve, killing any in-flight tween first. Gotcha worth telling in interviews: GSAP's *lag smoothing* slows its clock on weak GPUs — the glide stretched to ~10 s; `gsap.ticker.lagSmoothing(0)` fixes it (frames may skip, but time stays honest). **Learn:** tweening, easing curves, requestAnimationFrame.

3.6 Coordinate transforms — plan feet → world metres

The room beacons live in **plan feet** (origin bottom-left). The GLB is in metres; the viewer then scales and re-centres the model to fit the stage. So every beacon point goes through the same chain: feet × 0.3048 → metres, then × model scale, + model position. Getting one transform wrong puts the camera inside a wall — this is the single most classic 3D bug family. **Learn:** coordinate spaces, affine transforms (scale + translate), unit discipline.

4. Testing & deployment concepts

Golden tests (pytest): instead of asserting "code runs", the suite pins *real numbers* — plan X must yield envelope 38.4 × 65.5 ft, 10 snapped doors, 20 windows. Any future "improvement" that silently degrades a plan fails CI. This is regression testing by **golden master**. **Vitest** covers frontend math (boxUV, room-point transforms) — pure functions extracted precisely so they are testable without a browser.

Docker: the server ships as an image — "a box containing Python + exact dependencies" — so Render runs it identically to your laptop. The image is deliberately **slim (no torch, ~10× smaller)** because v1 scope cut the photo path. **Render** reads render.yaml ("blueprint" = infrastructure-as-code) and rebuilds on every git push; **Vercel** does the same for the frontend. Deploys are therefore reproducible and hands-off. **Learn:** images vs containers, Dockerfile stages, env vars, infrastructure-as-code, CI/CD idea.

5. What to learn, in order of payoff

Tier 1 — you use it every day here: Python (functions, exceptions, typing), HTTP + REST + status codes, React state/props/effects, git branching. **Tier 2 — the differentiators:** OpenCV morphology + connected components, computational geometry (dot products, projections, polygons), three.js scene graph + materials + lighting, async Python. **Tier 3 — round out the story:** Docker, CORS deeply, glTF format internals, GSAP/animation, testing strategy (golden masters, pure-function extraction). For each: be able to (a) define it in one sentence, (b) point to where Drishti uses it, (c) tell one bug story about it — that triple is exactly what interviews reward.

6. Ten interview questions you should own

Q: Walk me through your project in 60 seconds.

Architects share floor plans as CAD PDFs; non-technical people can't read them. Drishti turns the PDF into a walkable 3D building in the browser. A FastAPI backend reads the PDF's vector drawing commands with PyMuPDF, detects scale from dimension text by voting, builds wall masks with OpenCV morphology, finds rooms via connected components, and exports a named GLB with trimesh. A React + three.js frontend renders it with procedural PBR materials, real-scale sunlight, and room-by-room walk-inside navigation. 110 automated tests, deployed on Render + Vercel.

Q: Why parse vectors instead of using AI/computer vision on the image?

Determinism and accuracy. The PDF already contains exact coordinates and layer names — reading them gives envelope accuracy within 2.5% of client-confirmed dimensions, with explainable failures. An ML model (we evaluated CubiCasa) is the right tool for photos/scans, where no vectors exist — so that path exists behind the same API as a beta, and the architecture keeps it pluggable (perception.py is the seam).

Q: How do you find the real-world scale?

Dimension-text voting: regex every ft-in token, project each onto nearby parallel lines (dot product), and each text-line match votes line-length ÷ value = points-per-foot. Dominant vote wins, median of agreeing votes is the scale. Fallbacks: standard 12-inch column size, then user-supplied width.

Q: What was the hardest bug?

Rooms leaking to outside: every room connected to the exterior through hairline slits at doorways. Root cause — door strips were sized to a fat detection mask but the output mask's walls were eroded 2 px shorter, leaving 1-px gaps. Fix: overfill door strips 5 px into the output mask only. The lesson I quote: when two representations of the same thing coexist (two masks), every consumer must know which one it's reading.

Q: Why two masks at all?

The raster door-gap finder is hyper-sensitive to wall thickness — de-blobbing the mask halved door-snap counts. So: keep the exact old fat ellipse mask only for door detection, and a clean square-cornered rect mask for output. Accuracy of one feature was protected while improving another — a deliberate engineering trade-off.

Q: How does the server stay responsive under load?

Three layers: heavy parses run in a threadpool off the async event loop; a semaphore caps concurrent parses (extra requests queue rather than OOM); an asyncio.wait_for timeout turns hung work into a 504. Plus typed error codes and a GPU-OOM handler that empties the cache and returns 503.

Q: How do you texture models that have no UV coordinates?

CPU-side box projection: de-index the geometry, recompute flat normals, and per-triangle pick UVs from the two axes orthogonal to the dominant normal axis. It's triplanar mapping without shaders — exact for axis-aligned architecture, and upgrade-proof. Textures are drawn on canvases at runtime: zero downloads.

Q: How do you know it's accurate?

A golden test suite pins per-plan numbers — envelope, snapped doors, windows, rooms — for 4 real client sheets and a synthetic sample; the anchor is a client-confirmed envelope matched within ±2.5%. Any regression fails the suite. batch_eval.py prints the live scorecard.

Q: What would you do next / what are the limits?

Door snapping on dense multi-flat sheets (~36%) needs endpoint-first snapping; open-envelope sheets defeat room detection until we add envelope sealing; furniture and room labels are next features; photo path needs a GPU host; and a photoreal-render button via a hosted ControlNet API is researched for v1.1.

Q: Why FastAPI, React, three.js — justify the stack.

FastAPI: async, typed, automatic docs, and Python is where OpenCV/PyMuPDF/trimesh live. React+Vite: component model fits a stateful viewer UI, instant HMR. three.js via react-three-fiber: the standard browser 3D engine, declaratively composed. GLB between them is an open standard — the backend stays viewer-agnostic and the downloaded file works in Blender or any glTF viewer.

7. The most important code, annotated

Real excerpts from the repo (trimmed). Read each with its 'why' — that's the interview answer.

7.1 Scale by voting — server/pdf_vector.py :: _text_scale()

Projects each dimension text onto candidate lines; agreeing votes elect the scale.

```
FTIN = re.compile(r"^(\d+)['\u2019]-?(\d+)?[\"\u201d]?$")   # 12'-6"
for cx, cy, val in toks:                # every ft-in text token
    for ax, ay, dx, dy, L in lines:     # every drawn line
        t = ((cx-ax)*dx + (cy-ay)*dy) / (L*L)   # project text onto line
        if not (0.15 < t < 0.85):  continue      # must sit mid-line
        perp = math.hypot(cx-(ax+t*dx), cy-(ay+t*dy))
        if perp > 9:  continue                   # and hug it (points away)
        votes.append(round(L / val, 1))          # pt-per-ft vote
best = Counter(votes).most_common(1)[0][0]
close = [v for v in votes if abs(v - best) <= 0.3]
ppf = statistics.median(close)          # robust: outliers can't win
```

7.2 Heavy work, safely — server/main.py :: _run_heavy()

The three-layer shield: semaphore (queue, don't OOM) + threadpool (don't block the event loop) + timeout (hung → 504).

```
_INFER_SLOTS = asyncio.Semaphore(int(os.getenv("MAX_CONCURRENT_INFER", "1")))

async def _run_heavy(fn, *args, what="inference"):
    timeout = float(os.getenv("INFER_TIMEOUT_S", "120"))
    try:
        async with _INFER_SLOTS:                        # cap concurrency
            return await asyncio.wait_for(
                run_in_threadpool(fn, *args), timeout) # off the loop + clock
    except asyncio.TimeoutError:
        raise HTTPException(504, f"{what} timed out after {timeout:.0f}s")
    except Exception as e:
        if _is_gpu_oom(e):
            _free_gpu()
            raise HTTPException(503, "model ran out of GPU memory - retry")
        raise
```

7.3 Boot without the AI — server/main.py (lazy import = graceful degradation)

```
try:
    import perception            # needs torch + CubiCasa
    _PERCEPTION_ERR = None
except Exception as _e:          # slim deploy: absent ON PURPOSE
    perception = None            # server still boots, CAD path fully works
    _PERCEPTION_ERR = str(_e)
# later, on a photo upload:
if perception is None:
    raise HTTPException(503, BETA_MSG)   # friendly beta message, not a crash
```

7.4 CORS that can't be silently broken — server/main.py :: parse_origins()

```
def parse_origins(value, default="http://localhost:5173"):
    """ALLOWED_ORIGINS env -> list: comma, whitespace and
    trailing-slash tolerant (a trailing slash silently breaks CORS)."""
    out = []
    for o in (value or default).split(","):
        o = o.strip().rstrip("/")
        if o:
            out.append(o)
    return out or [default]
```

7.5 Triplanar UVs on the CPU — src/three/planMaterials.ts :: boxUV()

Per-face dominant-axis projection; also recomputes flat normals so masonry edges stay crisp.

```
export function boxUV(geo: THREE.BufferGeometry): THREE.BufferGeometry {
  const g = geo.index ? geo.toNonIndexed() : geo
  g.computeVertexNormals()            // flat per-face normals: crisp edges
  const p = g.attributes.position, n = g.attributes.normal
  const uv = new Float32Array(p.count * 2)
  for (let f = 0; f + 2 < p.count; f += 3) {
    const nx = Math.abs(n.getX(f)), ny = Math.abs(n.getY(f)), nz = Math.abs(n.getZ(f))
    for (let v = f; v < f + 3; v++) {
      let u: number, w: number
```

```
      if (ny >= nx && ny >= nz) { u = p.getX(v); w = p.getZ(v) } // floor/top
      else if (nx >= nz)        { u = p.getZ(v); w = p.getY(v) } // x-facing
      else                      { u = p.getX(v); w = p.getY(v) } // z-facing
      uv[v*2] = u; uv[v*2+1] = w
    }
  }
  g.setAttribute('uv', new THREE.BufferAttribute(uv, 2))
  return g
}
```

7.6 Plan feet → world metres — src/three/roomPoints.ts

The transform chain every beacon and camera glide relies on.

```
export const FT = 0.3048
export const EYE_FT = 5.2              // standing eye height

export function roomWorldPoint(room, f: FrameInfo, heightFt = EYE_FT) {
  return [
    f.position[0] + f.scale * room.x * FT,      // feet->m, then model
    f.position[1] + f.scale * heightFt * FT,    //   scale + position
    f.position[2] + f.scale * room.y * FT,      //   (same transform
  ]                                             //    Model.tsx applied)
}
// Gaze: toward the FARTHEST other beacon -> looks through doorways,
// not nose-first into the nearest wall.
```

7.7 Honest camera glides — src/components/CameraRig.tsx

```
gsap.ticker.lagSmoothing(0)   // low-fps GPUs: don't slow the clock —
                              // 0.8s glide stays 0.8s (frames may skip)
tween.current = gsap.to(o, {
  px: view.position[0], py: view.position[1], pz: view.position[2],
  tx: view.target[0],   ty: view.target[1],   tz: view.target[2],
  duration: 0.8, ease: 'power3.out',
  onUpdate() {                        // move camera AND orbit target
    camera.position.set(o.px, o.py, o.pz)      // together, every tick
    controls.target.set(o.tx, o.ty, o.tz)
    controls.update()
  },
})
```

7.8 No shared temp file — server/main.py :: /scene.glb

```
fd, out = tempfile.mkstemp(suffix=".glb", prefix="drishti_")
os.close(fd)                        # unique file per request (a fixed name
await run_in_threadpool(scene_to_glb.build_glb, s, out)   # let users
return FileResponse(out, media_type="model/gltf-binary",  # download each
    filename="scene.glb",                                 # other's model)
    background=BackgroundTask(os.remove, out))   # delete after sending
```

If you can explain sections 2.2, 2.6, 3.3 and tell the two-mask and hairline-slit bug stories from memory, you can defend this project in any interview. Good luck — you built something real.